



03

Python

What is Python?

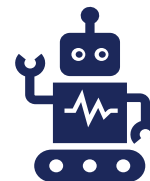
Python is a programming language widely used in **web applications**, **software development**, **data science**, and **machine learning** (ML). Developers use Python because it is efficient and easy to learn, and it can run on multiple platforms. Python software can be downloaded for free, integrates well with all types of systems, and increases development speed.



Server/Client web
development



Software
development



Data Science e
Machine Learning

What are the advantages of Python?

1. It has a basic syntax similar to the English language
2. It makes developers more productive, as fewer lines of code are required to achieve the same functionality compared to many other languages
3. It has an extensive standard library that contains reusable functionality for almost any task.
4. It can be easily integrated with other well-known programming languages such as Java, C, and C++
5. It has an active community of millions of developers
6. Many useful resources are available online for those who want to learn Python. For example, videos, tutorials, documentation, and developer guides can easily be found.

What is a library and what are Python libraries?

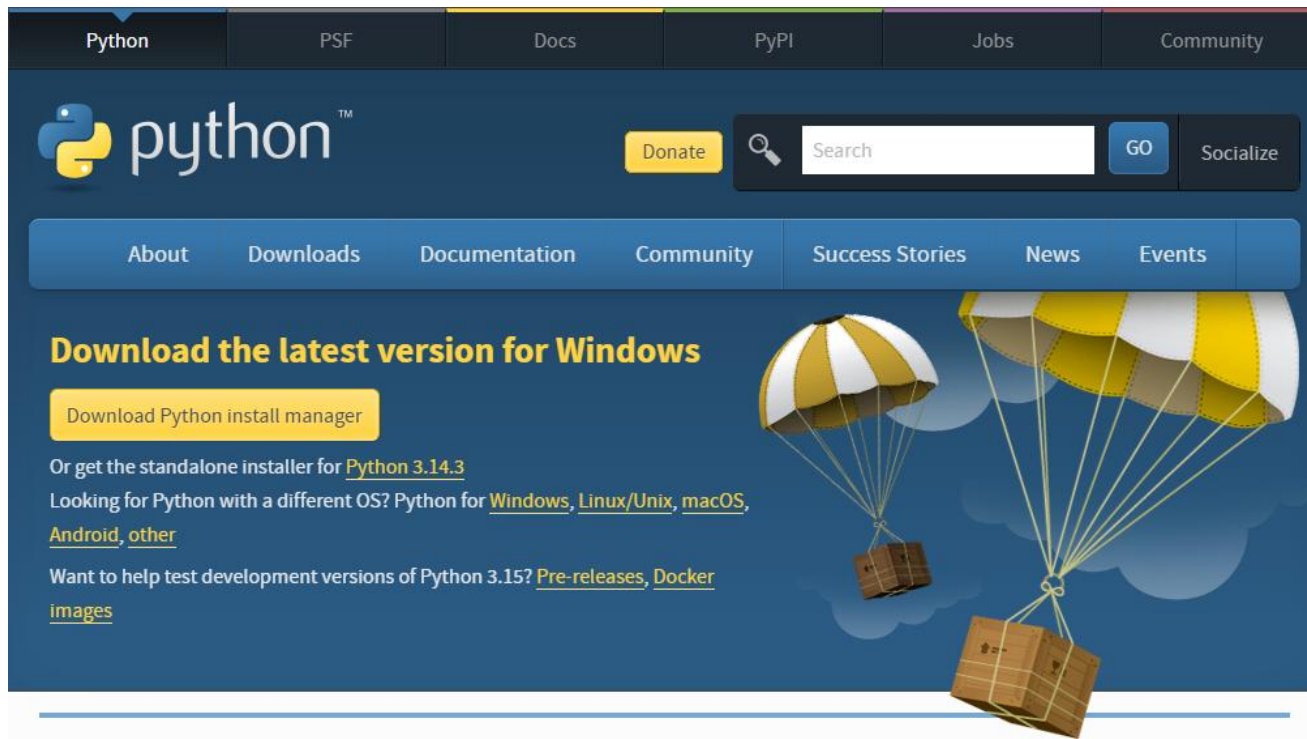
A library is a **collection of frequently used functions** that developers can include in their programs **to avoid writing code from scratch**.

By default, Python comes with the **standard library**, which contains a set of reusable functions. In addition, more than 200,000 Python libraries are available for various applications, including web development, data science, and machine learning.



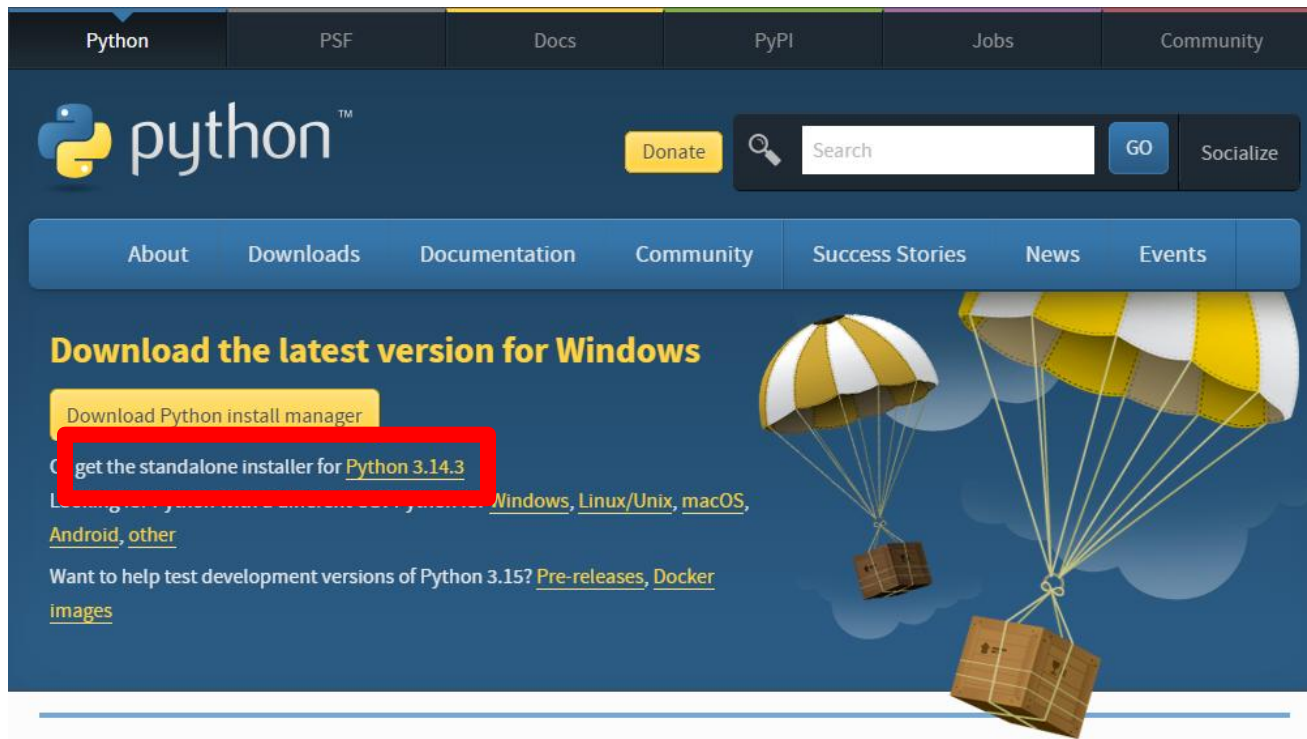
How to install Python

1. Go to the official website: <https://www.python.org/downloads/>



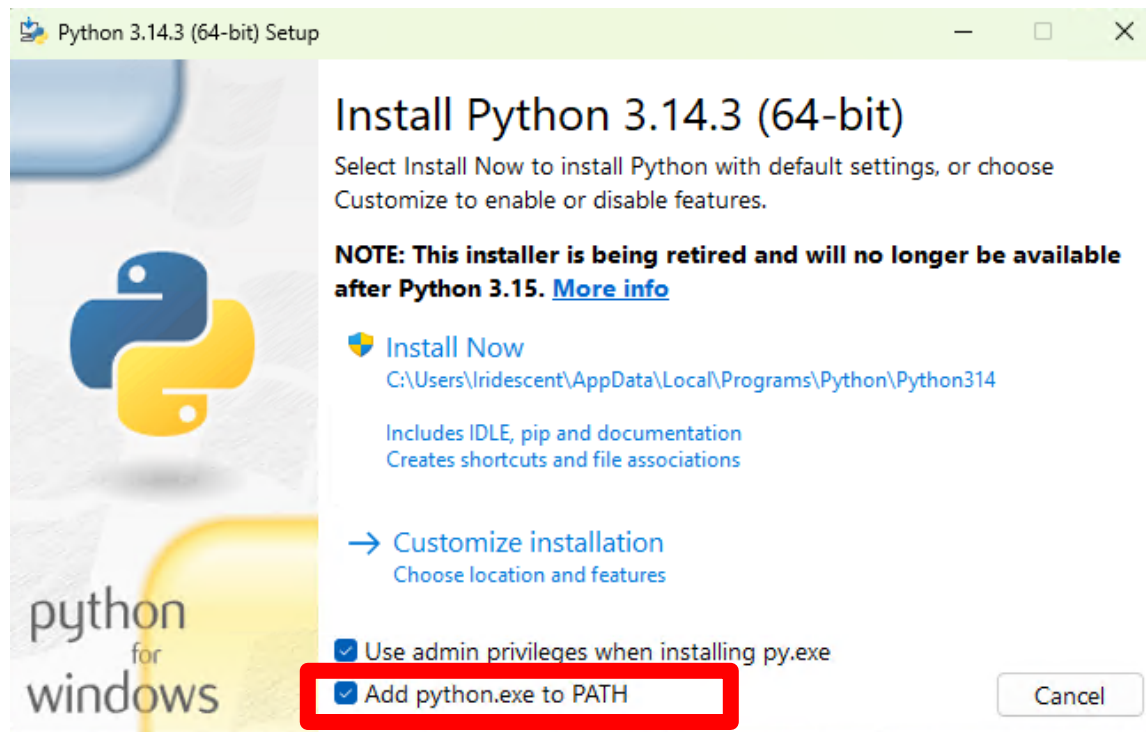
How to install Python

2. Click on "get the standalone installer for Python 3.x.y"



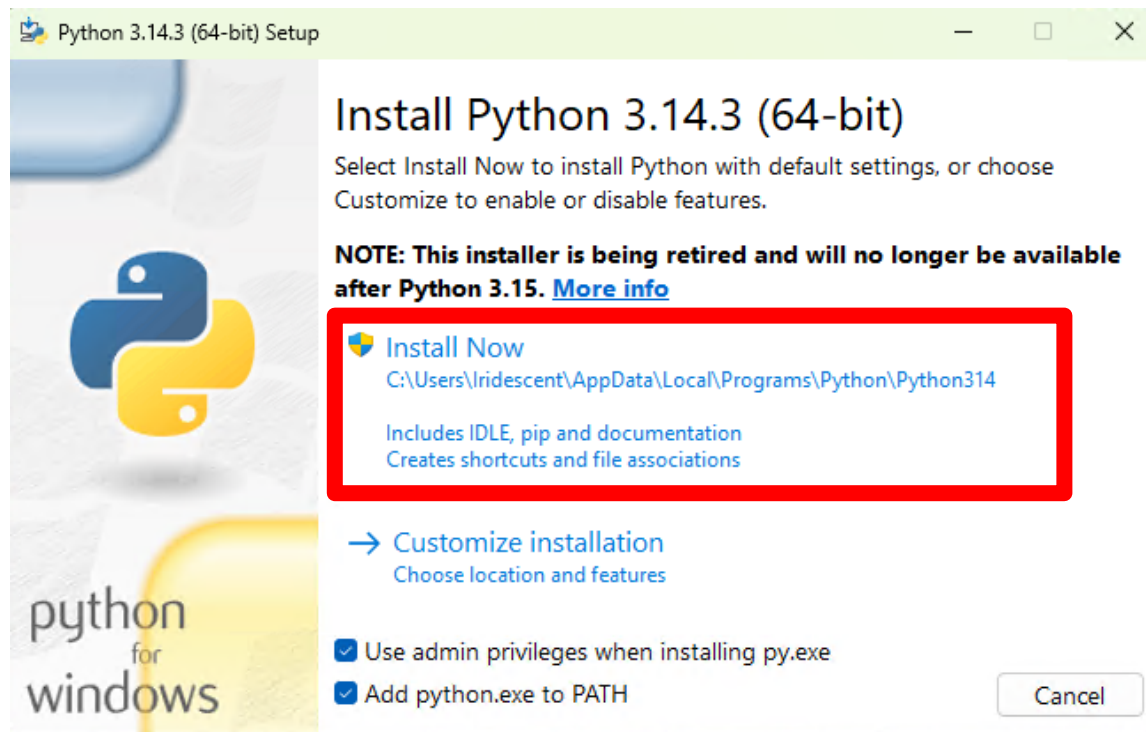
How to install Python

3. Launch the installer and select "Add python.exe to PATH"



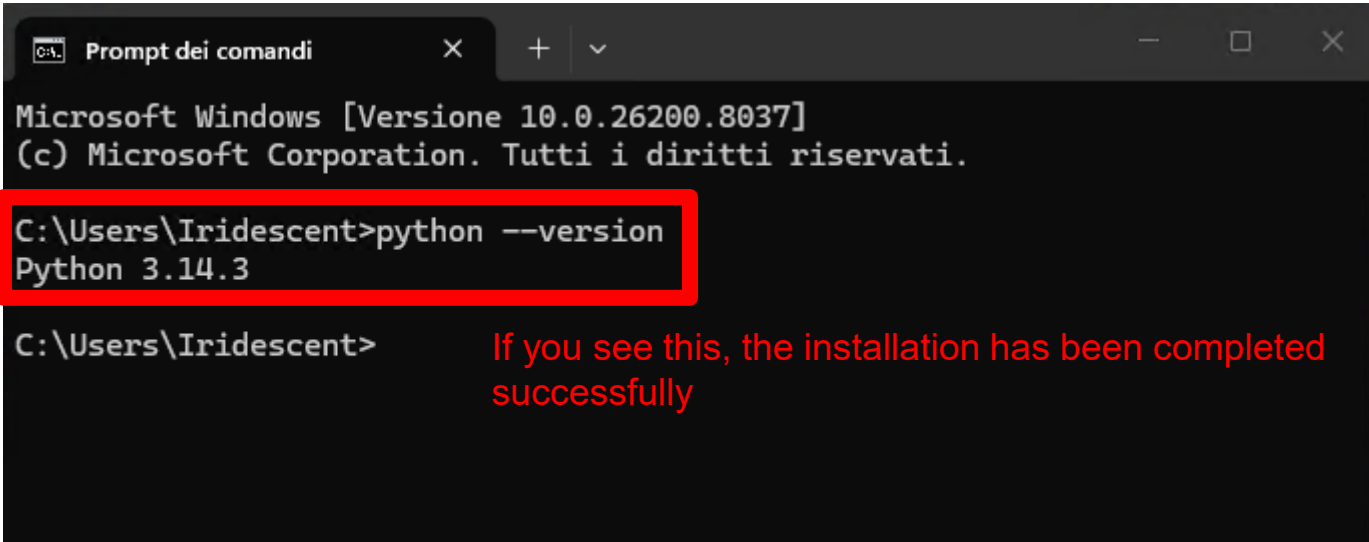
How to install Python

4. Click on "Install Now" and wait for the installation to complete



Check if installation was successful

1. Open a new terminal/command prompt (if you are use windows search for "cmd")
2. Type "`python --version`" and press enter



```
Prompt dei comandi
Microsoft Windows [Versione 10.0.26200.8037]
(c) Microsoft Corporation. Tutti i diritti riservati.
C:\Users\Iridescent>python --version
Python 3.14.3
C:\Users\Iridescent>
```

If you see this, the installation has been completed successfully

How to install libraries

- Libraries can be installed through command, the usual procedure is the following:
 1. Open a new terminal/command prompt (if you are use windows search for "cmd")
 2. Type "`pip install library_name`", where `library_name` is the name of the library to install, i.e. pandas
- If you want to install a specific version of a library library the command is: "`pip install library_name==version`", e.g. `pip install pandas==3.0.2`
- If you want to upgrade a library the command is "`pip --upgrade library_name`"
- If you want to uninstall a library the command is "`pip uninstall library_name`"

Visual Studio Code

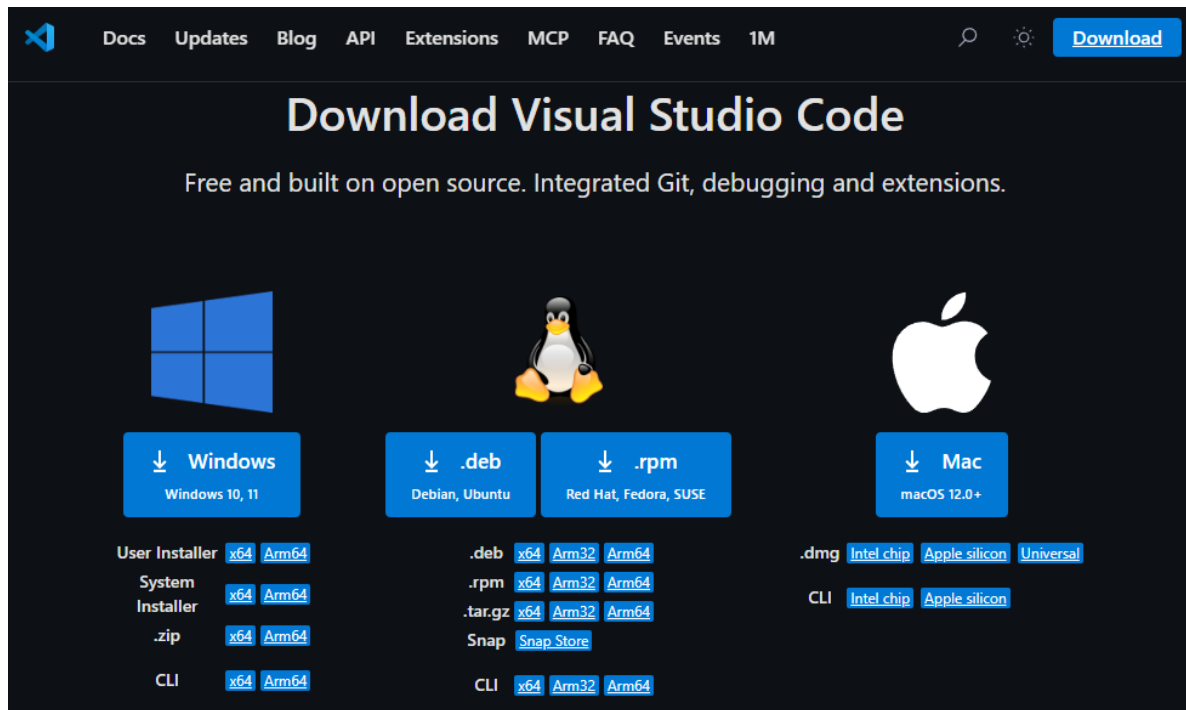
Visual Studio Code is a text editor developed by Microsoft.

Thanks to the various available **extensions**, VSCode can be used as a full-fledged IDE (Integrated Development Environment) with which you can build complete applications based on various languages and technologies, both frontend and web-oriented such as JavaScript, Angular, TypeScript, and React, as well as classic and non-classic backend solutions such as Java, Kotlin, and C.



How to install Visual Studio Code

1. Go to the official website: <https://code.visualstudio.com/Download>



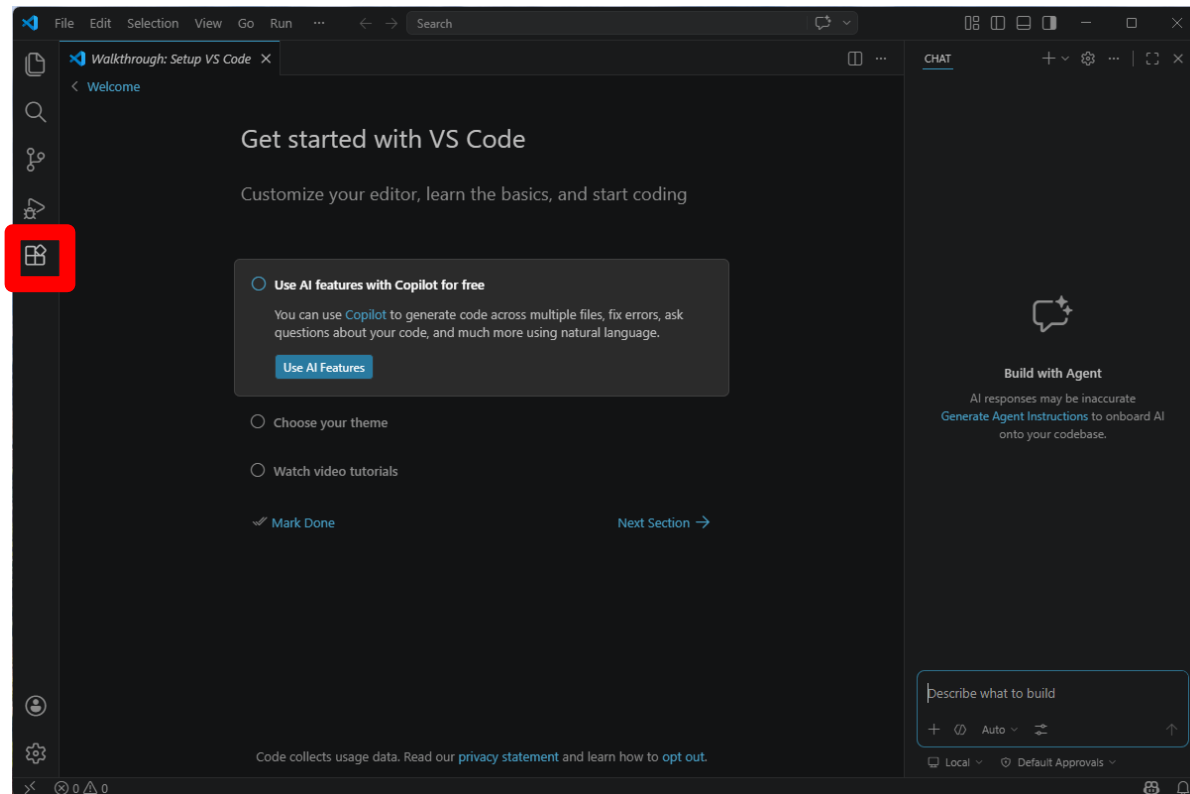
How to install Visual Studio Code

- 2. Select the correct version based on you OS



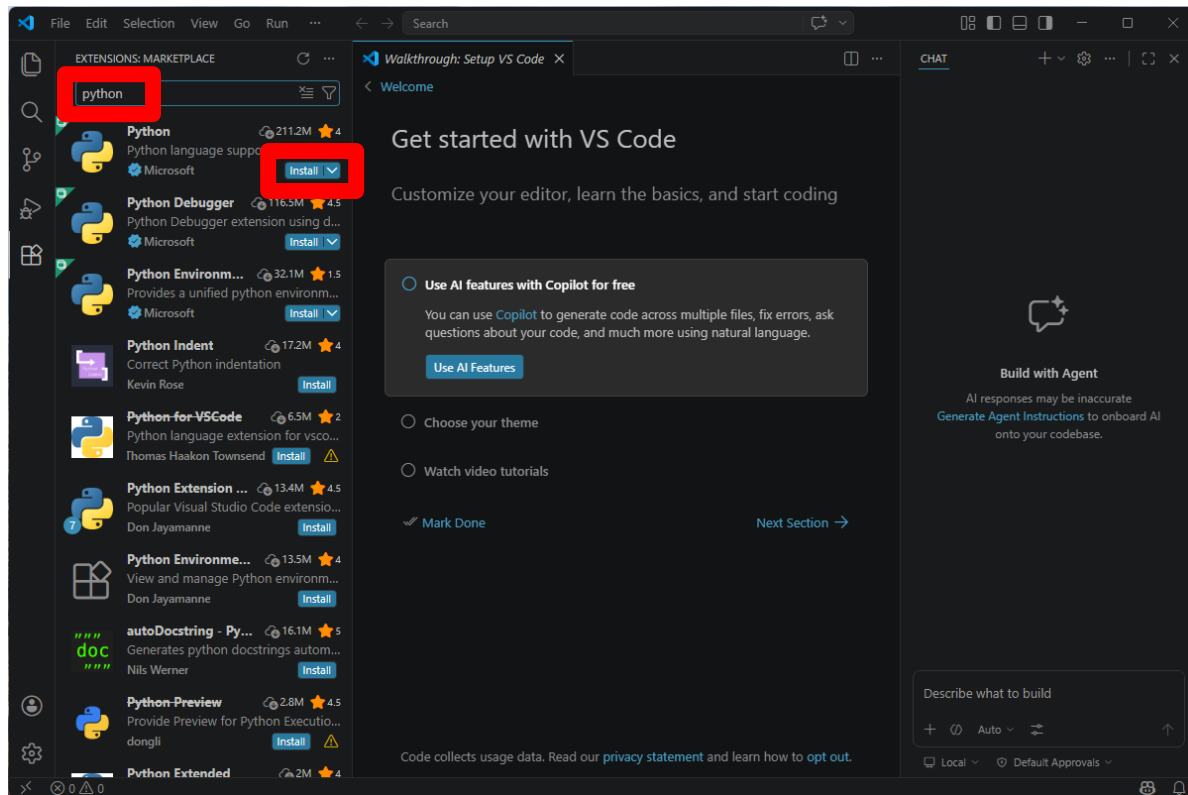
Setup Python in Visual Studio Code

1. After the installation, launch VSCode and click the extension icon



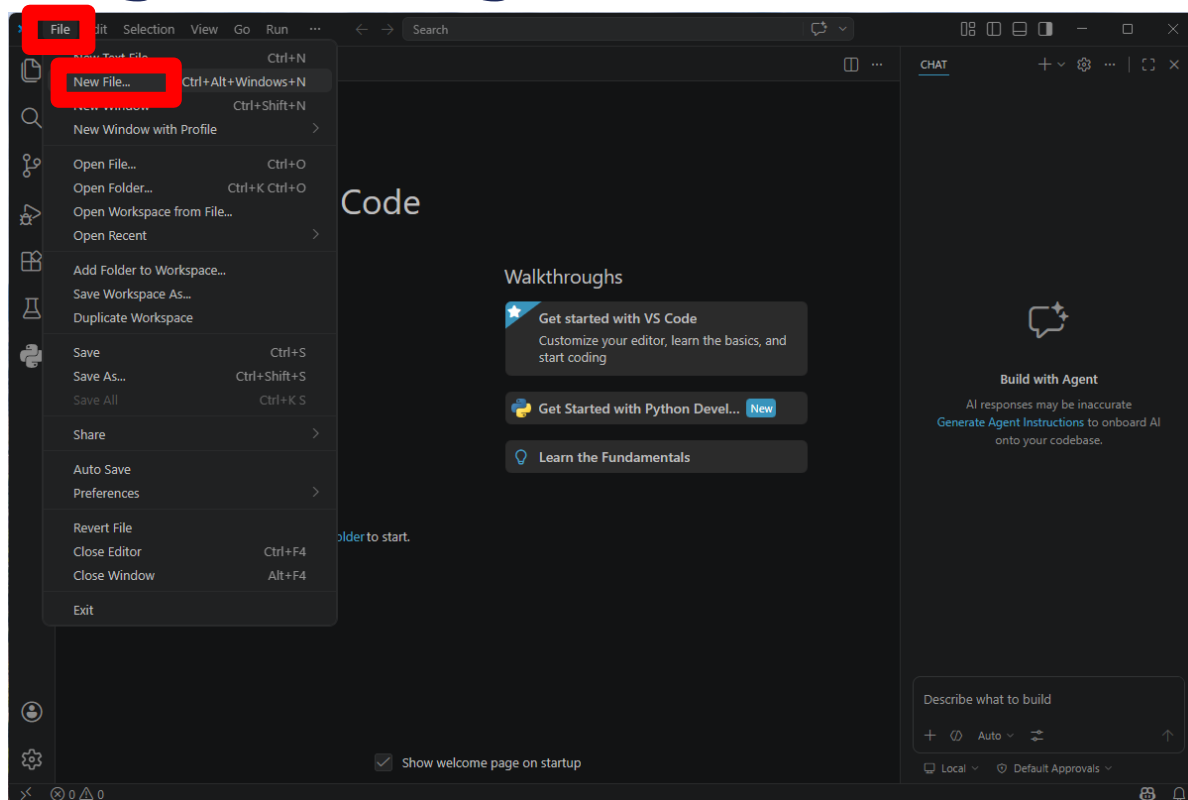
Setup Python in Visual Studio Code

1. After the installation, launch VSCode and click the extension icon
2. Search for «Python» and click install



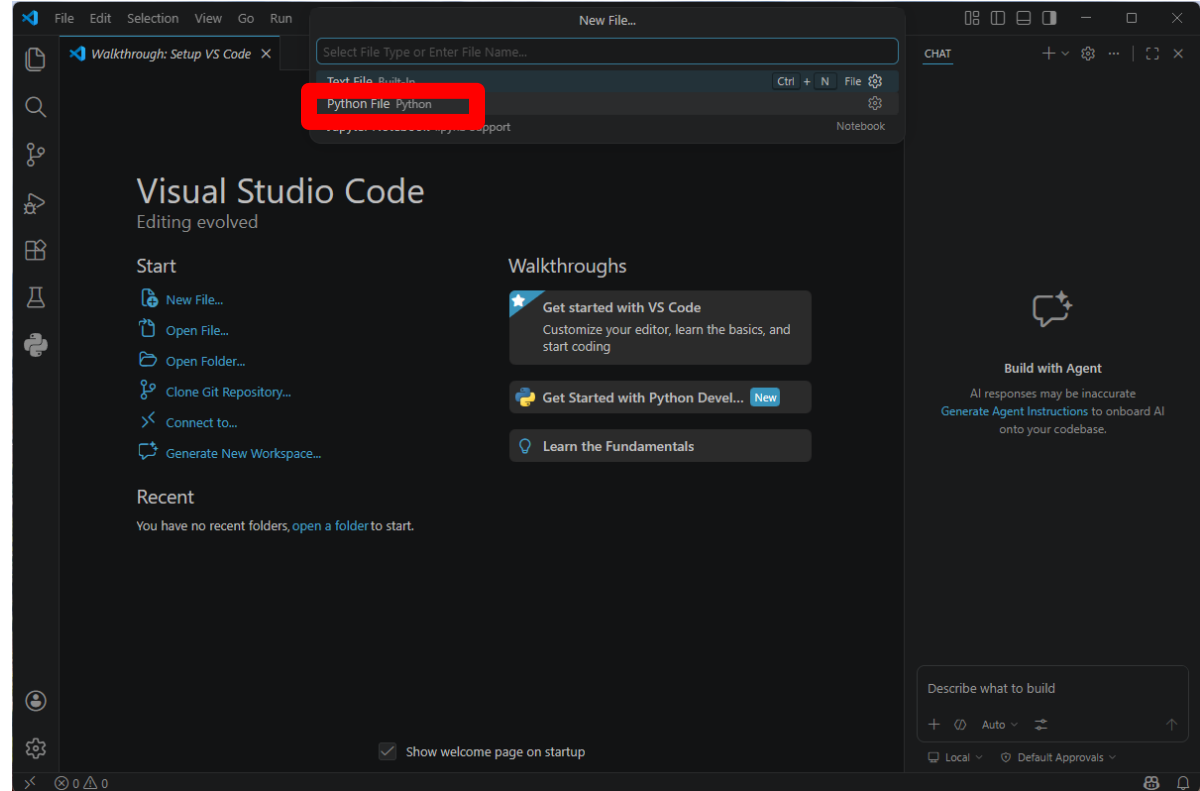
Check if everything is working

1. Click on file -> new file



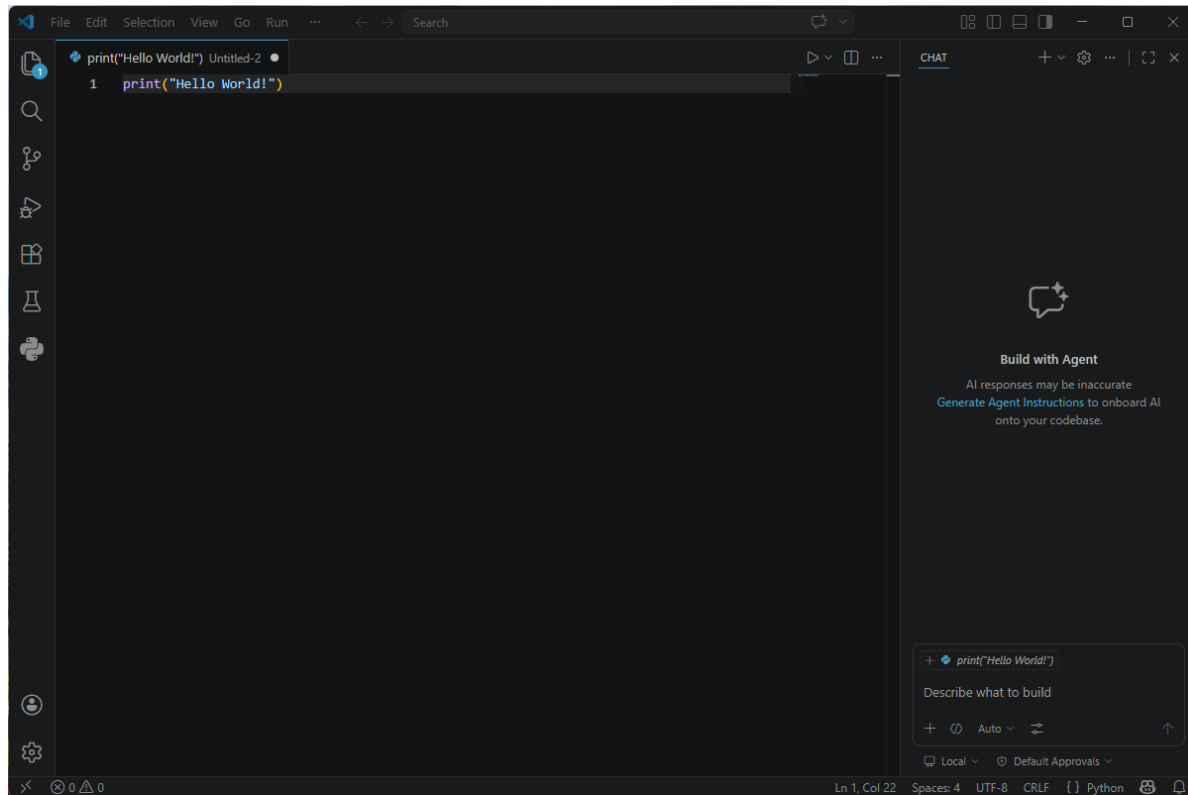
Check if everything is working

1. Click on file -> new file
2. Select «Python file»



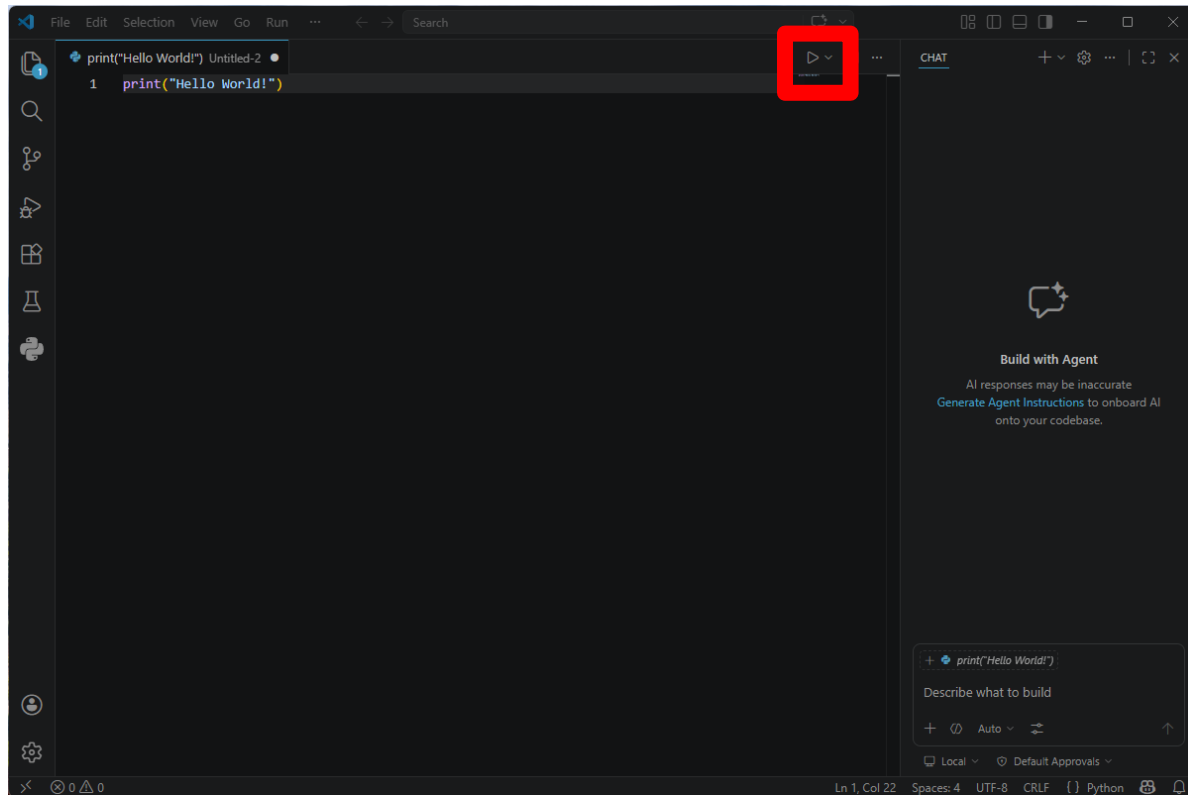
Check if everything is working

1. Click on file -> new file
2. Select «Python file»
3. Write your first «Hello World», typing:
`print("Hello World")`



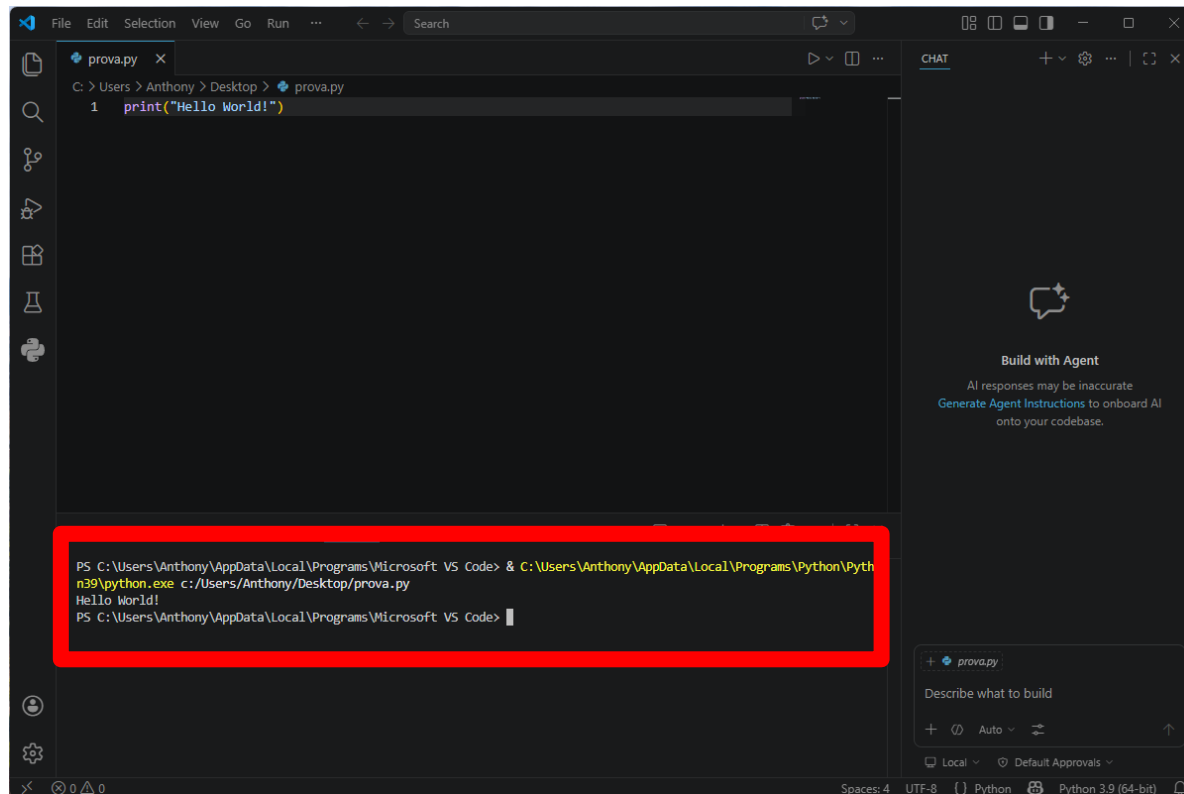
Check if everything is working

1. Click on file -> new file
2. Select «Python file»
3. Write your first «Hello World», typing:
`print("Hello World")`
4. Click Run File



Check if everything is working

1. Click on file -> new file
2. Select «Python file»
3. Write your first «Hello World», typing:
`print("Hello World")`
4. Click Run File
5. Check the output



```
File Edit Selection View Go Run ... Search
prova.py x
C: > Users > Anthony > Desktop > prova.py
1 print("Hello World!")

CHAT
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI
onto your codebase.

PS C:\Users\Anthony\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Anthony\AppData\Local\Programs\Python\Python39\python.exe c:/Users/Anthony/Desktop/prova.py
Hello World!
PS C:\Users\Anthony\AppData\Local\Programs\Microsoft VS Code> |

+ prova.py
Describe what to build
+ Auto -
Local Default Approvals
```

Python Fundamentals - Comments

```
 main.py  
1  #Commento in python  
2  
3  '''  
4  Commento  
5  in  
6  Python  
7  '''
```

Python Fundamentals - Variables

 main.py X

 main.py > ...

```
1  x = 10          # Intero
2  y = 3.14        # Numero decimale (float)
3  nome = "Anna"   # Stringa
4  logico = True   # Booleano
5
6  print(type(x))   # <class 'int'>
7  print(type(y))   # <class 'float'>
8  print(type(nome)) # <class 'str'>
9  print(type(logico)) # <class 'bool'>
10
```

Python Fundamentals – Variables

Multiple Assignment / Same Value Assignment

```
1  a, b, c = 1, 2.5, "Test"
2  print(a, b, c)  # Output: 1 2.5 Test
3
4
5  x = y = z = 0
6  print(x, y, z)  # Output: 0 0 0
```

```
1  numero = 5
2  numero = numero + 3  # Adesso numero vale 8
3
4  nome = "Mario"
5  cognome = "Rossi"
6  nome_completo = nome + " " + cognome
7  print(nome_completo)  # Output: Mario Rossi
```

Python Fundamentals – Lists

1. Create a list

```
main.py ×  
main.py > ...  
1  numeri = [1, 2, 3, 4, 5] # Lista di numeri interi  
2  nomi = ["Alice", "Bob", "Charlie"] # Lista di stringhe  
3  mista = [10, "Python", True, 3.14] # Lista con tipi diversi  
4
```

2. Accessing elements by index

```
main.py ×  
main.py > ...  
1  frutti = ["mela", "banana", "ciliegia"]  
2  print(frutti[0]) # Output: mela  
3  print(frutti[1]) # Output: banana  
4  print(frutti[-1]) # Output: ciliegia (ultimo elemento)  
5
```


Python Fundamentals – Lists

3. Slicing

main.py X

main.py > ...

```
1  numeri = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2
3  print(numeri[2:5])    # Output: [2, 3, 4] (dal 2° al 4° elemento)
4  print(numeri[:4])     # Output: [0, 1, 2, 3] (dall'inizio al 3° elemento)
5  print(numeri[5:])     # Output: [5, 6, 7, 8, 9] (dal 5° in poi)
6  print(numeri[::-2])   # Output: [0, 2, 4, 6, 8] (prende ogni 2 elementi)
7  print(numeri[::-1])   # Output: [9, 8, 7, ..., 0] (lista invertita)
8
```

Python Fundamentals – Lists

4. Change values

 main.py X

 main.py > ...

```
1 frutti = ["mela", "banana", "ciliegia"]
2 frutti[1] = "arancia"
3 print(frutti) # Output: ["mela", "arancia", "ciliegia"]
4
```

Python Fundamentals – Lists

5. Adding and Removing Elements

```
main.py X
main.py > ...
1  numeri = [1, 2, 3]
2  numeri.append(4)      # Aggiunge un elemento in fondo
3  print(numeri) # Output: [1, 2, 3, 4]
4
5  numeri.insert(1, 10)  # Inserisce 10 alla posizione 1
6  print(numeri) # Output: [1, 10, 2, 3, 4]
7
8  numeri.extend([5, 6]) # Aggiunge più elementi alla lista
9  print(numeri) # Output: [1, 10, 2, 3, 4, 5, 6]
10
11
12  numeri.remove(10) # Rimuove il primo valore 10
13  print(numeri) # Output: [1, 2, 3, 4, 5, 6]
14
15  ultimo = numeri.pop() # Rimuove e restituisce l'ultimo elemento
16  print(numeri) # Output: [1, 2, 3, 4, 5]
17  print(ultimo) # Output: 6
18
19  del numeri[0] # Rimuove l'elemento alla posizione 0
20  print(numeri) # Output: [2, 3, 4, 5]
21
22  numeri.clear() # Svuota la lista
23  print(numeri) # Output: []
24
```

Python Fundamentals – Lists

6. Checking if an element is in a list

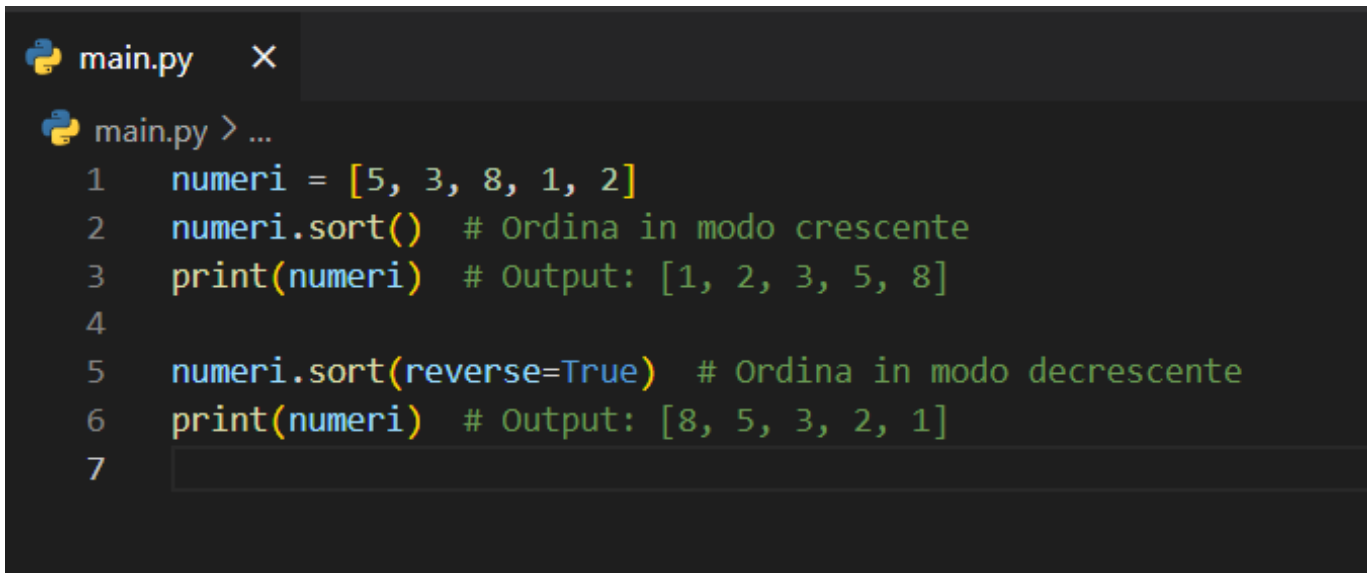
 main.py X

 main.py > ...

```
1 frutti = ["mela", "banana", "ciliegia"]  
2 print("mela" in frutti) # Output: True  
3 print("pera" not in frutti) # Output: True  
4
```

Python Fundamentals – Lists

7. Ordering a list

A screenshot of a Python IDE window titled 'main.py'. The code defines a list 'numeri' with values [5, 3, 8, 1, 2]. It then sorts the list in ascending order using 'numeri.sort()' and prints it, showing the output [1, 2, 3, 5, 8]. Next, it sorts the list in descending order using 'numeri.sort(reverse=True)' and prints it, showing the output [8, 5, 3, 2, 1].

```
main.py  X  
main.py > ...  
1  numeri = [5, 3, 8, 1, 2]  
2  numeri.sort()  # Ordina in modo crescente  
3  print(numeri)  # Output: [1, 2, 3, 5, 8]  
4  
5  numeri.sort(reverse=True)  # Ordina in modo decrescente  
6  print(numeri)  # Output: [8, 5, 3, 2, 1]  
7
```

Python Fundamentals - Conditions

Conditions in Python allow blocks of code to be executed only if certain conditions are true. The main construct for conditions in Python is if-elif-else.

```
main.py X
main.py > [x] x
1 x = 10
2
3 if x > 5:
4     print("x è maggiore di 5") # ✅ Viene eseguito perché 10 > 5
5
```

```
main.py X
main.py > ...
1 x = 3
2
3 if x > 5:
4     print("x è maggiore di 5")
5 else:
6     print("x è minore o uguale a 5") # ✅ Questa parte viene eseguita
7
```

Python Fundamentals - Indentation

```
main.py > ...  
1  x = 10  
2  
3  if x > 5:  
4      print("x è maggiore di 5") # Indentato con 4 spazi  
5  print("Questa riga è fuori dal blocco if") # Non indentato, eseguito sempre  
6
```



```
7  
8  if True:  
9      print("Errore!") # ✗ Questo darà un errore di indentazione  
10
```



```
11  
12  if True:  
13      print("Indentazione corretta!")  
14
```

Python Fundamentals - Conditions

Conditions in Python allow blocks of code to be executed only if certain conditions are true. The main construct for conditions in Python is if-elif-else.

```
main.py  X
main.py > ...
1  x = 7
2
3  if x > 10:
4      print("x è maggiore di 10")
5  elif x > 5:
6      print("x è maggiore di 5 ma minore o uguale a 10") # ✅ Questa viene eseguita
7  else:
8      print("x è minore o uguale a 5")
9
```


Python Fundamentals - Conditions

Conditions in Python allow blocks of code to be executed only if certain conditions are true. The main construct for conditions in Python is if-elif-else.

```
main.py X
main.py > ...
1 x = 7
2 if x > 5 and x < 10:
3     print("x è compreso tra 5 e 10") # ✅ Viene eseguito perché entrambe le condizioni sono vere
4
5 x = 3
6 if x < 5 or x > 10:
7     print("x è minore di 5 o maggiore di 10") # ✅ Viene eseguito perché 3 < 5
8
9 x = 10
10 if not x < 5:
11     print("x non è minore di 5") # ✅ Viene eseguito perché "x < 5" è falso
12
13
```

Python Fundamentals - Conditions

Conditions in Python allow blocks of code to be executed only if certain conditions are true. The main construct for conditions in Python is if-elif-else.

 main.py

 main.py > ...

```
1  frutta = ["mela", "banana", "arancia"]
2
3  if "mela" in frutta:
4      print("La mela è nella lista") # ✓ Viene eseguito
5
6  if "kiwi" not in frutta:
7      print("Il kiwi non è nella lista") # ✓ Viene eseguito
```

Python Fundamentals - Loops

Loops in Python allow you to repeat an operation multiple times without having to write the code manually. We have two kind of loops:

1. while:

 main.py X main.py > ...

```
1   x = 0
```

```
2
```

```
3   while x < 5:
```

```
4       print(f"x è {x}")
```

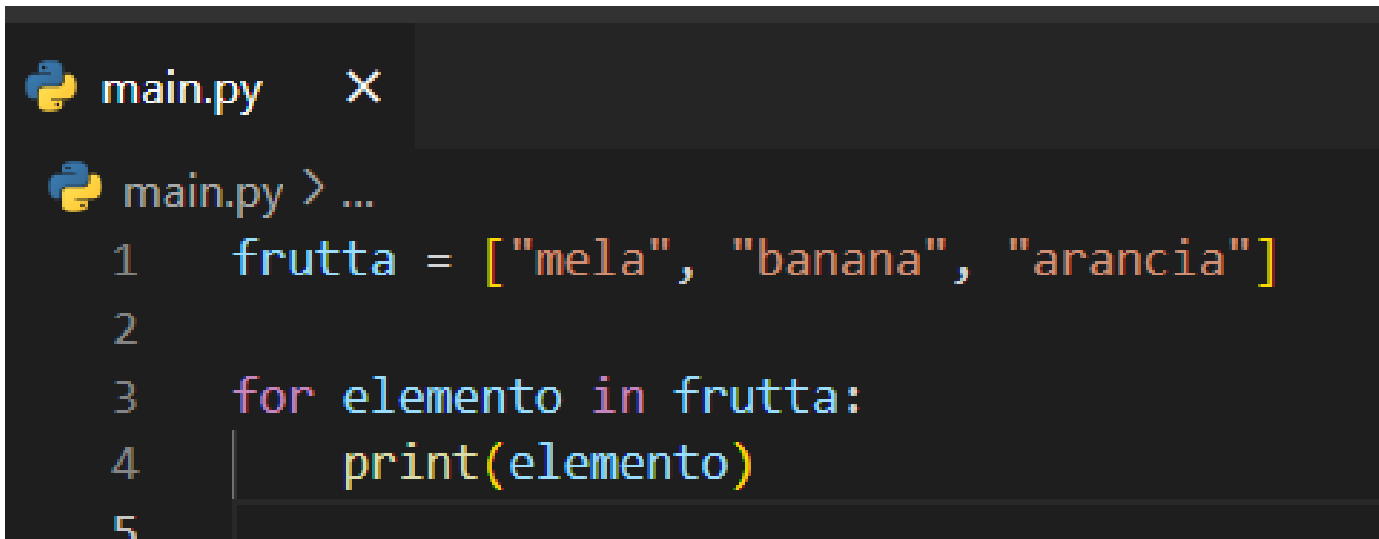
```
5       x += 1  # Incremento per evitare un ciclo infinito
```

```
6
```

Python Fundamentals - Loops

Loops in Python allow you to repeat an operation multiple times without having to write the code manually. We have two kind of loops:

2. for:

A screenshot of a code editor window with a dark background. The window title is 'main.py' with a close button 'X'. The code is written in Python and shows a list 'frutta' containing 'mela', 'banana', and 'arancia'. A 'for' loop iterates over this list, printing each element. The code is as follows:

```
main.py > ...  
1  frutta = ["mela", "banana", "arancia"]  
2  
3  for elemento in frutta:  
4      print(elemento)  
5
```

Python Fundamentals – Set

1. Set creation:

```
main.py ×  
main.py > ...  
1  numeri = {1, 2, 3, 4, 5}  
2  print(numeri) # Output: {1, 2, 3, 4, 5} (ordine casuale)  
3  
4  numeri = {1, 2, 2, 3, 4, 4}  
5  print(numeri) # Output: {1, 2, 3, 4}  
6  
7  set_vuoto = set() # ✓ Questo è un set vuoto  
8
```

Python Fundamentals – Set

2. Adding elements:

```
main.py ×  
main.py > ...  
1  frutta = {"mela", "banana"}  
2  frutta.add("arancia")  
3  print(frutta)  # Output: {'mela', 'banana', 'arancia'}
```

3. Removing elements:

```
6  frutta.remove("mela")  # Se l'elemento non esiste, dà errore  
7  frutta.discard("banana")  # Non dà errore se l'elemento non esiste  
8  print(frutta)  # Output: {'arancia'}
```

Python Fundamentals – Set

4. Set operations

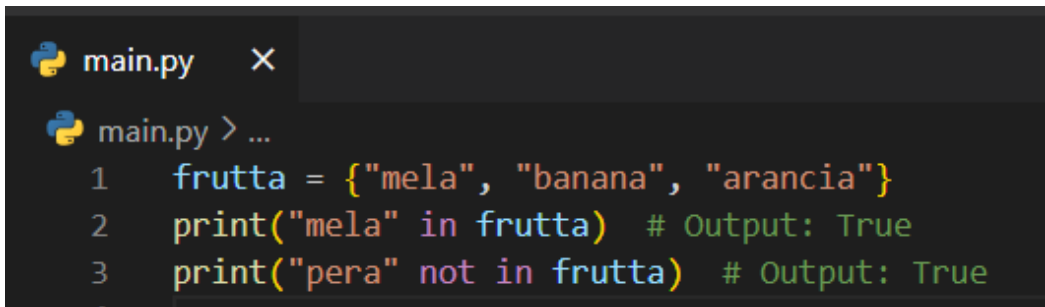
main.py ×

main.py > ...

```
1  A = {1, 2, 3}
2  B = {3, 4, 5}
3  print(A | B) # Output: {1, 2, 3, 4, 5}
4  print(A.union(B)) # Output: {1, 2, 3, 4, 5}
5
6  print(A & B) # Output: {3}
7  print(A.intersection(B)) # Output: {3}
8
9  print(A - B) # Output: {1, 2} (elementi in A ma non in B)
10 print(A.difference(B)) # Output: {1, 2}
11
12 print(A ^ B) # Output: {1, 2, 4, 5} (elementi in A o B, ma non in entrambi)
13 print(A.symmetric_difference(B)) # Output: {1, 2, 4, 5}
```

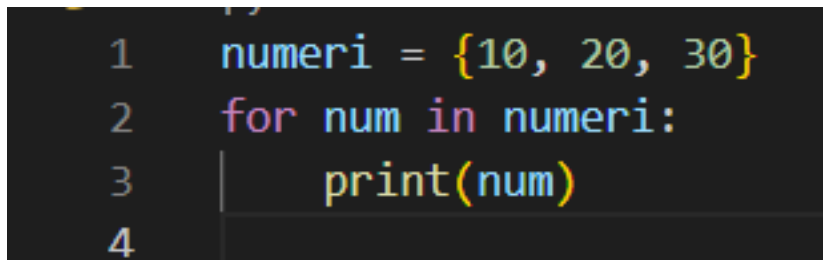
Python Fundamentals – Set

5. Checking for the presence of an element



```
main.py X
main.py > ...
1  frutta = {"mela", "banana", "arancia"}
2  print("mela" in frutta) # Output: True
3  print("pera" not in frutta) # Output: True
```

6. Iterating over a Set



```
1  numeri = {10, 20, 30}
2  for num in numeri:
3      print(num)
4
```


Python Fundamentals – Dictionary

1. Dictionary creation

```
main.py X
main.py > ...
1  studente = {
2      "nome": "Alice",
3      "età": 22,
4      "città": "Roma"
5  }
6  print(studente)
7  # Output: {'nome': 'Alice', 'età': 22, 'città': 'Roma'}
```

```
8
9  studente = dict(nome="Alice", età=22, città="Roma")
10 print(studente)
11
```

Python Fundamentals – Dictionary

2. Accessing values

```
12 print(studente["nome"]) # Output: Alice
13 print(studente.get("età")) # Output: 22
```

3. Changing a value

```
15 studente["età"] = 23 # Modifica il valore della chiave "età"
16 print(studente)
17 # Output: {'nome': 'Alice', 'età': 23, 'città': 'Roma'}
```

4. Adding a new key-value pair

```
9 studente["università"] = "Sapienza"
0 print(studente)
1 # Output: {'nome': 'Alice', 'età': 23, 'città': 'Roma', 'università': 'Sapienza'}
```

Python Fundamentals – Dictionary

5. Removing elements

```
24 del studente["città"]
25 print(studente)
26 # Output: {'nome': 'Alice', 'età': 23, 'università': 'Sapienza'}
27
28 studente.clear()
29 print(studente) # Output: {}
```

Python Fundamentals – Dictionary

6. Iterating over a dictionary

main.py ×

main.py > ...

```
1     studente = {"nome": "Alice", "età": 22, "città": "Roma"}
2
3     for chiave in studente:
4         print(chiave, ":", studente[chiave])
```

```
6     for chiave, valore in studente.items():
7         print(f"{chiave}: {valore}")
```

Python Fundamentals – Dictionary

6. Iterating over a dictionary

```
0 print(studente.keys())  
1 # Output: dict_keys(['nome', 'età', 'città'])  
2  
3 print(studente.values())  
4 # Output: dict_values(['Alice', 22, 'Roma'])  
5
```

Python Fundamentals – Dictionary

7. Nested Dictionaries

```
7  studenti = {  
8      "studente1": {"nome": "Alice", "età": 22},  
9      "studente2": {"nome": "Bob", "età": 24}  
10 }  
11  
12 print(studenti["studente1"]["nome"]) # Output: Alice  
13
```

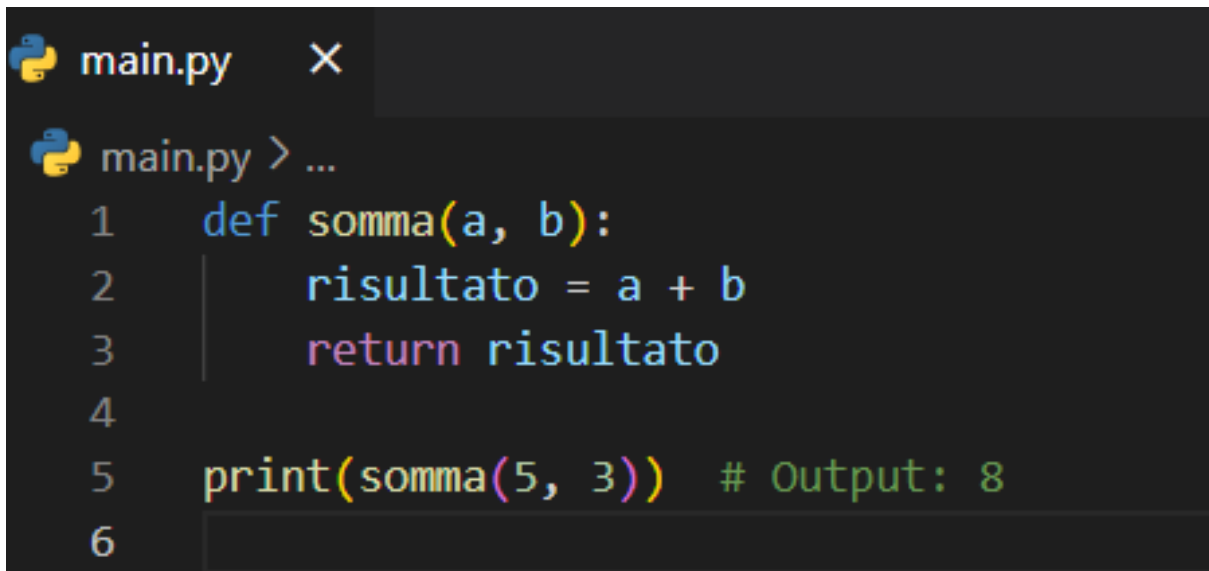
Python Fundamentals – Functions

Functions in Python allow you to reuse code, improve organization, and make the program more readable. A function is a reusable block of code, defined with `def`, that can accept parameters and return a value.

```
 main.py > ...  
1  def welcome():  
2      print("Ciao, benvenuto in Python!")  
3  
4  welcome()  # Chiamata alla funzione  
5
```

Python Fundamentals – Functions

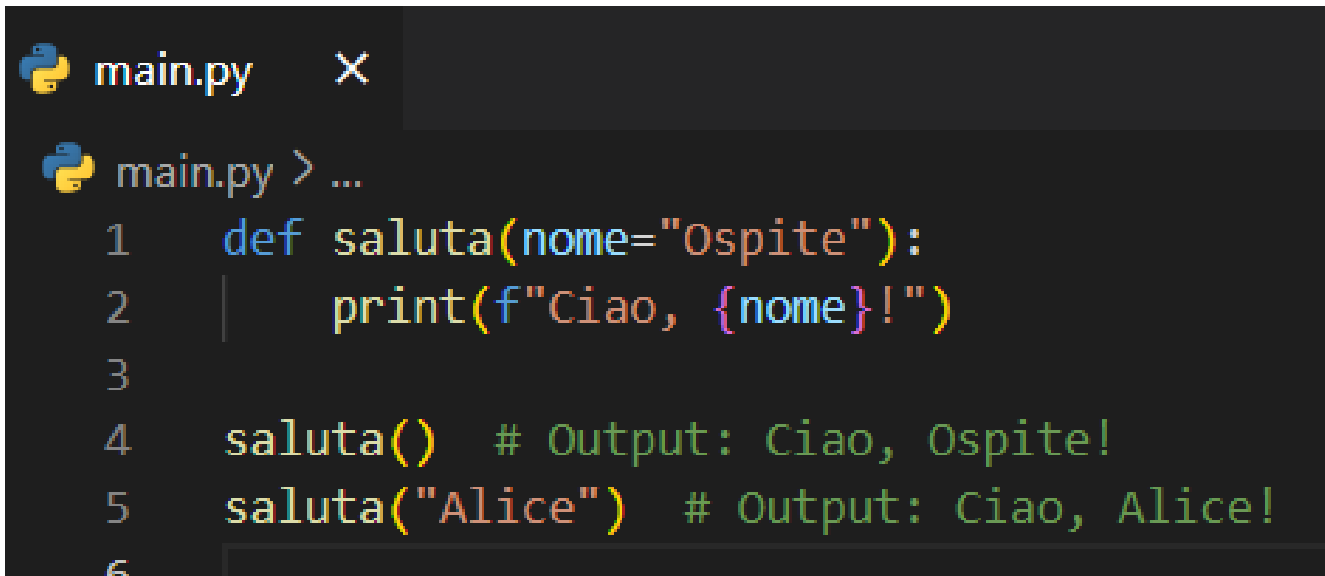
This is a function with two parameters that returns their sum



```
main.py  X  
  
main.py > ...  
1  def somma(a, b):  
2      risultato = a + b  
3      return risultato  
4  
5  print(somma(5, 3))  # Output: 8  
6
```


Python Fundamentals – Functions

If the parameter “name” is not defined, it will be automatically set to “Ospite”.



```
main.py X
main.py > ...
1  def saluta(nome="Ospite"):
2      print(f"Ciao, {nome}!")
3
4  saluta() # Output: Ciao, Ospite!
5  saluta("Alice") # Output: Ciao, Alice!
6
```

Python Fundamentals – Variables

Global and local variables

```
main.py > ...  
1  x = 10  # Variabile globale  
2  
3  def funzione():  
4      y = 5  # Variabile locale  
5      print(x + y)  
6  
7  funzione()  
8  # print(y)  # Errore: y non è accessibile fuori dalla funzione  
9
```

Numpy

NumPy is a fundamental library for scientific computing in Python. It provides support for multidimensional arrays, fast mathematical operations, and tools for linear algebra and statistics.



Numpy – How to install and import

1. To install, open a terminal or prompt and write
`"pip install numpy"`
you have to execute this command only once
2. To import write in your code
`"import numpy as np"`



Numpy – How to create an array from a list

```
1  import numpy as np
2
3  lista = [1, 2, 3, 4]
4  arr = np.array(lista)
5  print(arr)  # Output: [1 2 3 4]
6  print(type(arr))  # Output: <class 'numpy.ndarray'>
```

```
1  import numpy as np
2
3  matrice = np.array([[1, 2, 3], [4, 5, 6]])
4  print(matrice)
5
6  '''
7  [[1 2 3]
8   | [4 5 6]]
9  '''
```

Numpy – Special arrays

```
1  import numpy as np
2
3  np.zeros((3, 3))    # Matrice 3x3 di zeri
4  np.ones((2, 2))     # Matrice 2x2 di uni
5  np.eye(3)           # Matrice identità 3x3
6  np.full((2, 3), 5)  # Matrice 2x3 piena di 5
7
```

Numpy – ranges

1. `arange(start, stop, step)`
2. `linspace(start, stop, num)`. Divides an interval into num parts.

```
1  import numpy as np
2
3  np.arange(1, 10, 2) # Output: array([1, 3, 5, 7, 9])
4  np.linspace(0, 1, 5) # Output: array([0. , 0.25, 0.5 , 0.75, 1. ])
5
```

Numpy – arrays properties

```
1  import numpy as np
2
3  arr = np.array([[1, 2, 3], [4, 5, 6]])
4
5  print(arr.shape)  # Output: (2, 3) → Dimensione (righe, colonne)
6  print(arr.ndim)   # Output: 2 → Numero di dimensioni
7  print(arr.size)   # Output: 6 → Numero totale di elementi
8  print(arr.dtype)  # Output: int64 → Tipo di dati degli elementi
```


Numpy – Mathematical and Matrix Operations

```
1 import numpy as np
2
3 a = np.array([1, 2, 3])
4 b = np.array([4, 5, 6])
5
6 print(a + b) # Output: [5 7 9]
7 print(a - b) # Output: [-3 -3 -3]
8 print(a * b) # Output: [4 10 18] (moltiplicazione elemento per elemento)
9 print(a / b) # Output: [0.25 0.4 0.5]
10 print(a ** 2) # Output: [1 4 9] (elevamento a potenza)
11
```

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 B = np.array([[5, 6], [7, 8]])
5
6 print(np.dot(A, B)) # Moltiplicazione matriciale
7 print(A @ B)       # Altro modo per la moltiplicazione matriciale
8 print(A.T)         # Trasposta della matrice A
9
```

Numpy – Indexing and slicing

```
1  import numpy as np
2
3  arr = np.array([10, 20, 30, 40])
4  print(arr[0]) # Output: 10
5  print(arr[-1]) # Output: 40
```

```
1  import numpy as np
2
3  arr = np.array([0, 1, 2, 3, 4, 5])
4  print(arr[1:4]) # Output: [1 2 3] (dal secondo al quarto elemento)
5  print(arr[:3]) # Output: [0 1 2] (primi tre elementi)
6  print(arr[::-1]) # Output: [5 4 3 2 1 0] (array invertito)
```

Numpy – Statistical and Universal Functions

```
1  import numpy as np
2
3  arr = np.array([1, 2, 3, 4, 5])
4
5  print(np.mean(arr))    # Media → Output: 3.0
6  print(np.median(arr))  # Mediana → Output: 3.0
7  print(np.std(arr))     # Deviazione standard → Output: 1.41
8  print(np.sum(arr))     # Somma degli elementi → Output: 15
9  print(np.max(arr))     # Valore massimo → Output: 5
10 print(np.min(arr))     # Valore minimo → Output: 1
11
```

```
1  import numpy as np
2
3  arr = np.array([0, np.pi/2, np.pi])
4  print(np.sin(arr))    # Calcola il seno degli elementi
5  print(np.log(arr + 1)) # Logaritmo naturale (evita log(0))
6  print(np.exp(arr))    # Esponenziale
```

Numpy – Write and read files

```
1  import numpy as np
2
3  arr = np.array([0, np.pi/2, np.pi])
4
5  np.save("dati.npy", arr) # Salva un array
6  arr_caricato = np.load("dati.npy") # Carica un array
7  print(arr_caricato)
```

Pandas

Pandas is one of the most powerful libraries for data analysis, manipulation, and visualization in Python. It is widely used in Data Science, Machine Learning, and data analytics.

Pandas



Pandas – How to install and import

1. To install, open a terminal or prompt and write
`"pip install pandas"`
you have to execute this command only once
2. To import write in your code
`"import pandas as pd"`

Pandas



Pandas – Series

```
1  import pandas as pd
2
3  s = pd.Series([10, 20, 30, 40])
4  print(s)
```

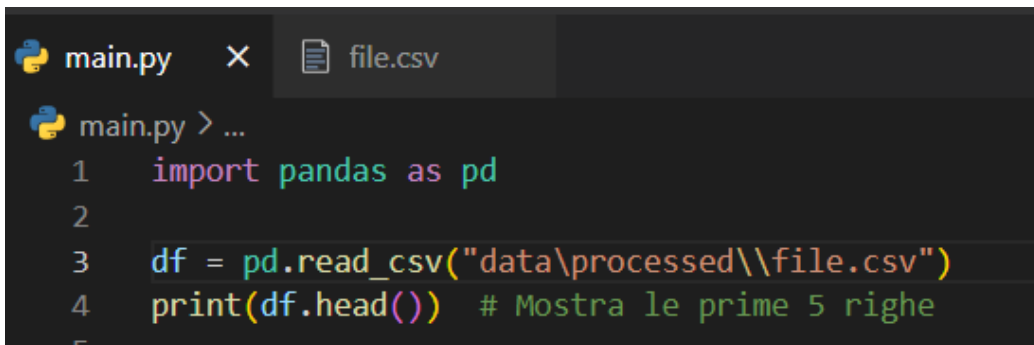
```
y
0    10
1    20
2    30
3    40
dtype: int64
```

Pandas – Dataframes

```
1 import pandas as pd
2
3 dati = {
4     "Nome": ["Alice", "Bob", "Carlo"],
5     "Età": [25, 30, 35],
6     "Salario": [50000, 60000, 70000]
7 }
8
9 df = pd.DataFrame(dati)
10 print(df)
```

	Nome	Età	Salario
0	Alice	25	50000
1	Bob	30	60000
2	Carlo	35	70000

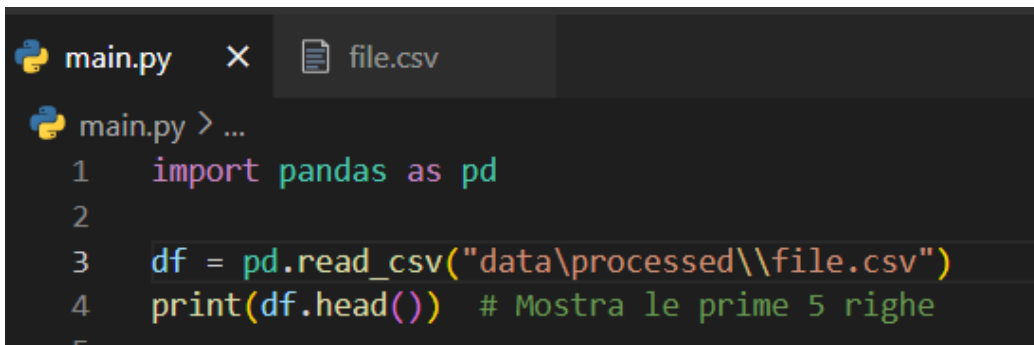
Pandas – Creating a DataFrame from an external source



```
main.py  ×  file.csv  
main.py > ...  
1  import pandas as pd  
2  
3  df = pd.read_csv("data\processed\file.csv")  
4  print(df.head()) # Mostra le prime 5 righe  
5
```

	A	B	C
0	12	2	3
1	43	5	0
2	3	4	11

Pandas – Creating a DataFrame from an external source



```
main.py  ×  file.csv  
main.py > ...  
1  import pandas as pd  
2  
3  df = pd.read_csv("data\processed\file.csv")  
4  print(df.head()) # Mostra le prime 5 righe  
5
```

	A	B	C
0	12	2	3
1	43	5	0
2	3	4	11

Pandas – Creating a DataFrame from an external source

Parameter	Description	Example
sep=";"	Specify the separator (e.g. ; for European CSV files)	<code>pd.read_csv("file.csv", sep=";")</code>
header=None	Indicates that the file has no headers	<code>pd.read_csv("file.csv", header=None)</code>
names=["col1", "col2"]	Assigns custom column names	<code>pd.read_csv("file.csv", names=["Nome", "Età"])</code>
index_col=0	Uses the first column as the index	<code>pd.read_csv("file.csv", index_col=0)</code>
usecols=["Nome", "Età"]	Selects only certain columns	<code>pd.read_csv("file.csv", usecols=["Nome", "Età"])</code>
nrows=10	Reads only the first n rows	<code>pd.read_csv("file.csv", nrows=10)</code>
skiprows=2	Skips the first n rows	<code>pd.read_csv("file.csv", skiprows=2)</code>
na_values=["NA", "null"]	Replaces missing values with NaN	<code>pd.read_csv("file.csv", na_values=["NA", "null"])</code>

Pandas – Explore dataframe

Dimensions and info

```
1  import pandas as pd
2
3  df = pd.read_excel(r"data\\processed\\file.csv")
4
5  print(df.shape) # Output: (righe, colonne)
6  print(df.info()) # Info sulle colonne e sui tipi di dati
7  print(df.describe()) # Statistiche generali sulle colonne numeriche
8
```

Pandas – Explore dataframe

Accessing to columns and rows

```
1  import pandas as pd
2
3  df = pd.read_excel(r"data\\processed\\file.csv")
4
5  print(df["Nome"]) # Seleziona una colonna
6  print(df[["Nome", "Salario"]]) # Seleziona più colonne
7
8  print(df.iloc[1]) # Seleziona la riga con indice 1
9  print(df.loc[0, "Nome"]) # Seleziona un valore specifico
```

Pandas – Filter Data

```
1 import pandas as pd
2
3 df = pd.read_excel(r"data\\processed\\file.csv")
4
5 df_filtro = df[df["Età"] > 28] # Filtra chi ha più di 28 anni
6 print(df_filtro)
7
```

```
1 import pandas as pd
2
3 df = pd.read_excel(r"data\\processed\\file.csv")
4
5 df_filtro = df[(df["Età"] > 28) & (df["Salario"] > 55000)]
6 print(df_filtro)
7
```

Pandas – Modify data

1. Add a column

```
1 import pandas as pd
2
3 df = pd.read_excel(r"data\\processed\\file.csv")
4
5 df["Bonus"] = df["Salario"] * 0.10
```

2. Delete a column

```
1 import pandas as pd
2
3 df = pd.read_excel(r"data\\processed\\file.csv")
4
5 df.drop(columns=["Bonus"], inplace=True)
```

Pandas – Modify data

3. Change the values

```
1  import pandas as pd
2
3  df = pd.read_excel(r"data\\processed\\file.csv")
4
5  df.loc[df["Nome"] == "Alice", "Salario"] = 52000
6
```

4. Delete a row

```
1  import pandas as pd
2
3  df = pd.read_excel(r"data\\processed\\file.csv")
4
5  df.drop(index=0, inplace=True) # Elimina la prima riga
6
```


Pandas – Save data

```
df.to_csv("dati_puliti.csv", index=False)
```

```
15 df.to_excel("dati.xlsx", sheet_name="Foglio1",  
16 index=False)
```

Matplotlib

Matplotlib is the main library for creating charts and visualizations in Python. It allows you to create line charts, bar charts, histograms, scatter plots, and much more.



Matplotlib – How to install and import

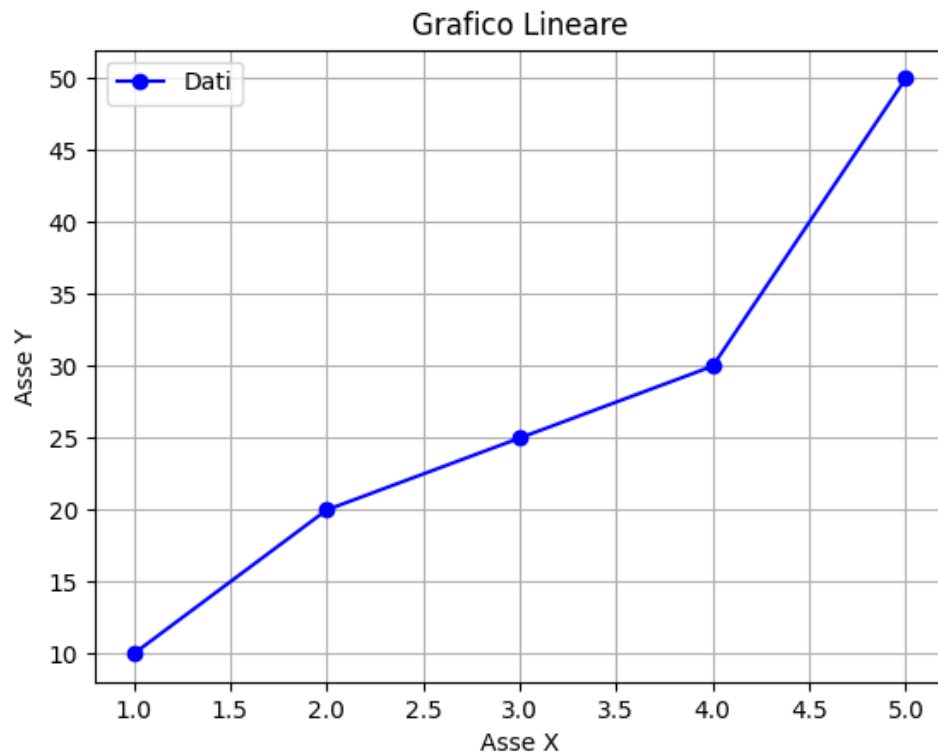
1. To install, open a terminal or prompt and write
`"pip install matplotlib"`
you have to execute this command only once
2. To import write in your code
`"import matplotlib.pyplot as plt"`



Matplotlib – Create a chart

```
1  import matplotlib.pyplot as plt
2
3  x = [1, 2, 3, 4, 5] # Ascisse
4  y = [10, 20, 25, 30, 50] # Ordinate
5
6  plt.plot(x, y, marker="o", linestyle="-", color="b", label="Dati") # Crea il grafico
7  plt.xlabel("Asse X") # Etichetta asse X
8  plt.ylabel("Asse Y") # Etichetta asse Y
9  plt.title("Grafico Lineare") # Titolo del grafico
10 plt.legend() # Mostra la legenda
11 plt.grid(True) # Aggiunge la griglia
12 plt.show() # Mostra il grafico
```

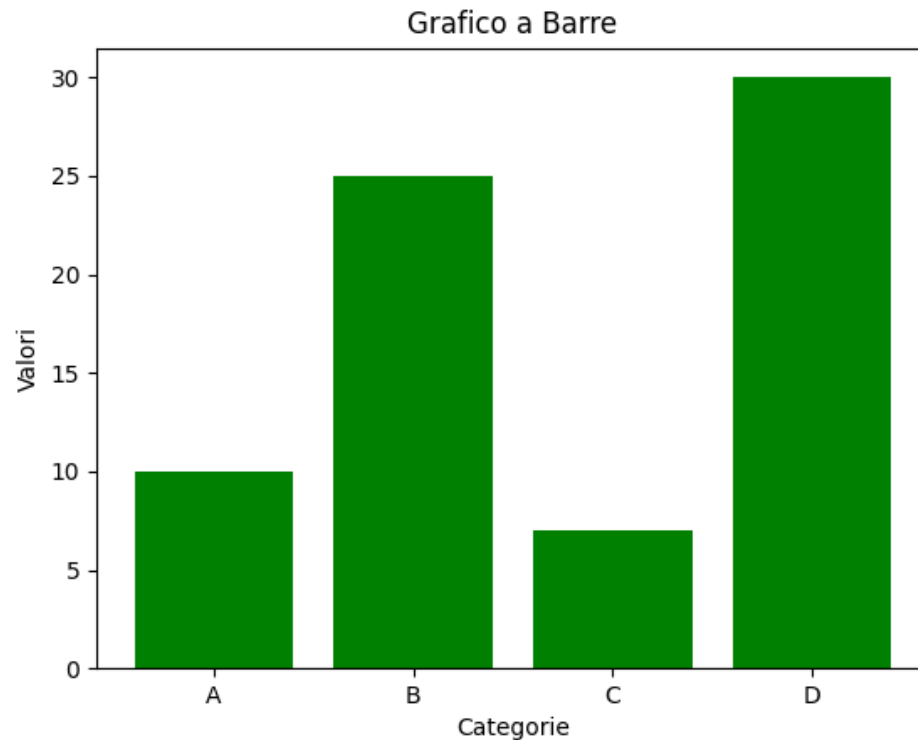
Matplotlib – Create a chart



Matplotlib – Bar Chart

```
1  import matplotlib.pyplot as plt
2
3  x = ["A", "B", "C", "D"]
4  y = [10, 25, 7, 30]
5
6  plt.bar(x, y, color="green")
7  plt.xlabel("Categorie")
8  plt.ylabel("Valori")
9  plt.title("Grafico a Barre")
10 plt.show()
11
```

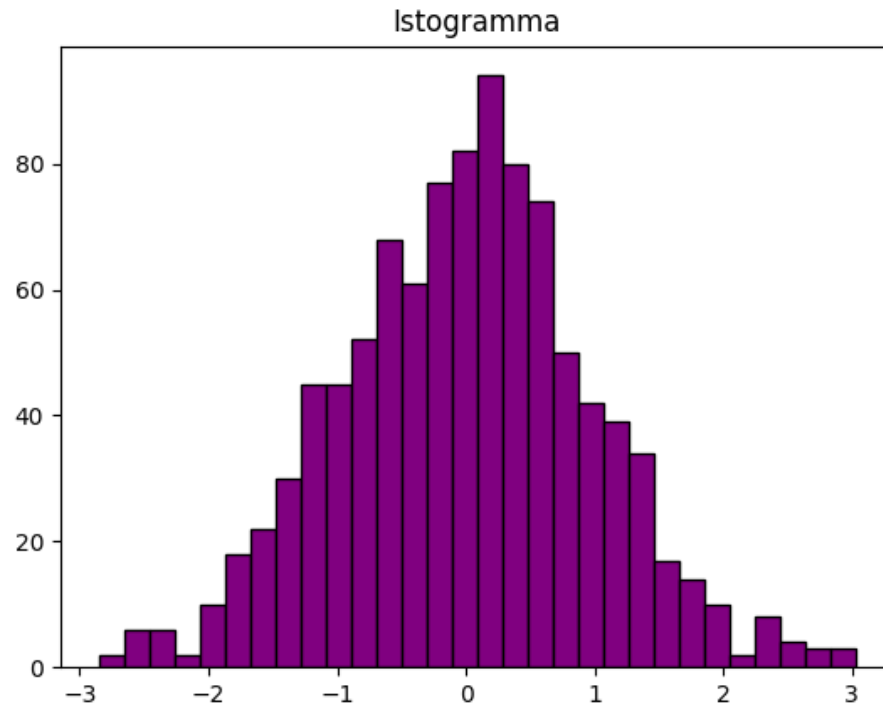
Matplotlib – Bar Chart



Matplotlib – Histogram

```
1  import matplotlib.pyplot as plt
2
3  import numpy as np
4
5  dati = np.random.randn(1000) # Genera 1000 numeri casuali
6  plt.hist(dati, bins=30, color="purple", edgecolor="black")
7  plt.title("Istogramma")
8  plt.show()
```

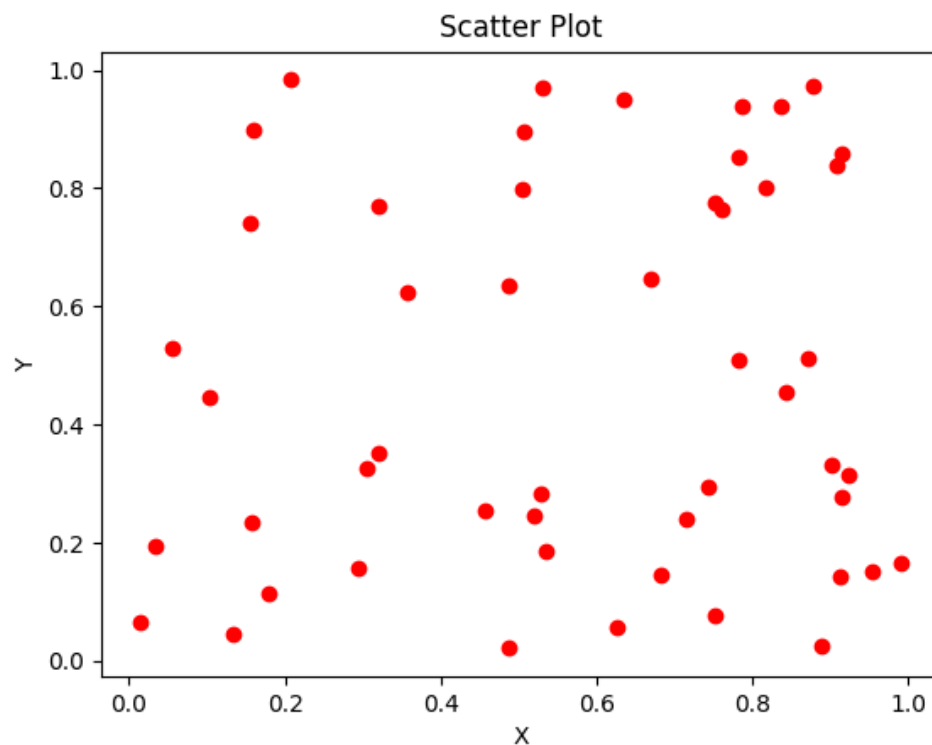

Matplotlib – Histogram



Matplotlib – Scatter Plot

```
1  import matplotlib.pyplot as plt
2
3  import numpy as np
4
5  x = np.random.rand(50)
6  y = np.random.rand(50)
7
8  plt.scatter(x, y, color="red")
9  plt.xlabel("X")
10 plt.ylabel("Y")
11 plt.title("Scatter Plot")
12 plt.show()
13
```

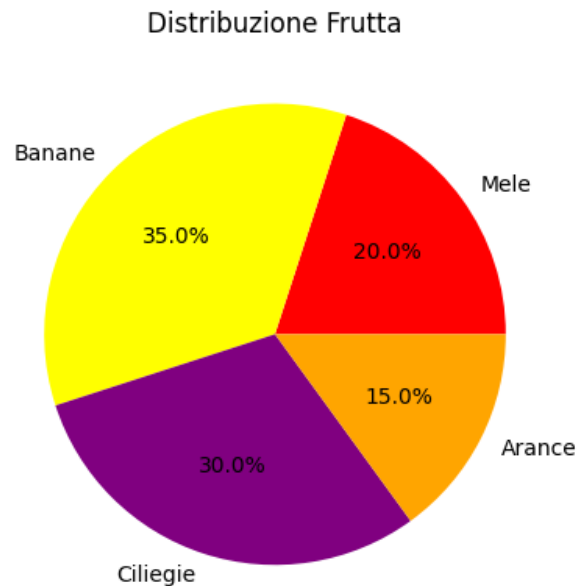
Matplotlib – Scatter Plot



Matplotlib – Pie Chart

```
main.py  X
main.py > ...
1  import matplotlib.pyplot as plt
2
3  labels = ["Mele", "Banane", "Ciliegie", "Arance"]
4  valori = [20, 35, 30, 15]
5
6  plt.pie(valori, labels=labels, autopct="%1.1f%%", colors=["red", "yellow", "purple", "orange"])
7  plt.title("Distribuzione Frutta")
8  plt.show()
9
```

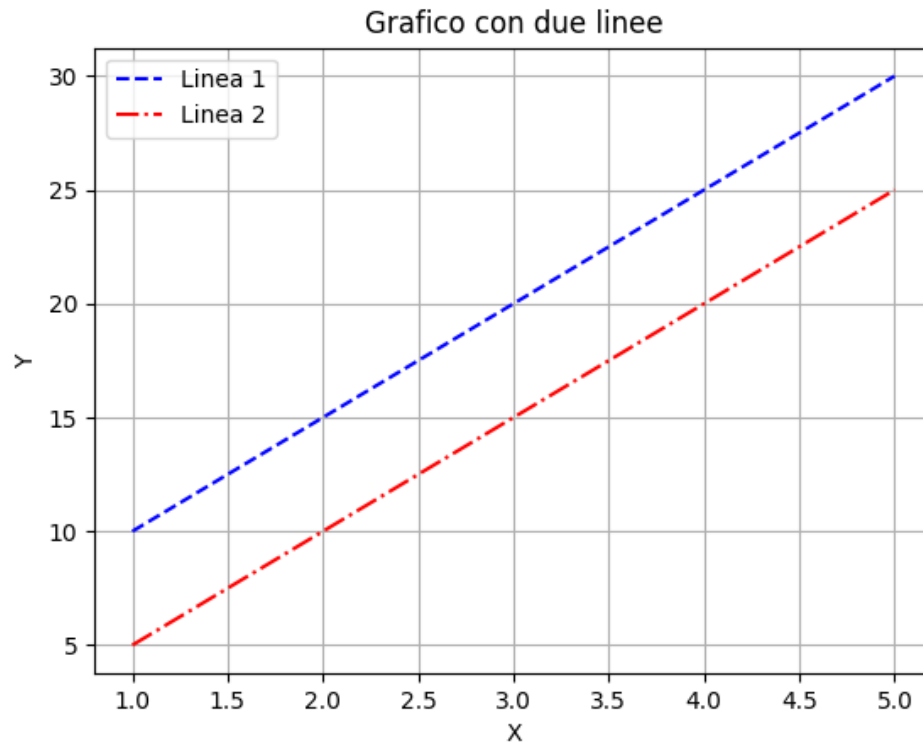
Matplotlib – Pie Chart



Matplotlib – Adding different lines

```
main.py X
main.py > ...
1  import matplotlib.pyplot as plt
2
3  x = [1, 2, 3, 4, 5]
4  y1 = [10, 15, 20, 25, 30]
5  y2 = [5, 10, 15, 20, 25]
6
7  plt.plot(x, y1, label="Linea 1", color="blue", linestyle="--")
8  plt.plot(x, y2, label="Linea 2", color="red", linestyle="-.")
9
10 plt.xlabel("x")
11 plt.ylabel("y")
12 plt.title("Grafico con due linee")
13 plt.legend()
14 plt.grid(True)
15 plt.show()
16
```

Matplotlib – Adding different lines



Matplotlib – Subplots

```
1  import matplotlib.pyplot as plt
2
3  x = [1, 2, 3, 4, 5]
4  y1 = [10, 15, 20, 25, 30]
5  y2 = [5, 10, 15, 20, 25]
6
7  fig, axs = plt.subplots(2, 2, figsize=(10, 6)) # 2x2 griglia di grafici
8
9  # Primo grafico
10 axs[0, 0].plot(x, y1, color="blue")
11 axs[0, 0].set_title("Grafico Lineare")
12
13 # Secondo grafico
14 axs[0, 1].bar(x, y1, color="green")
15 axs[0, 1].set_title("Grafico a Barre")
16
17 # Terzo grafico
18 axs[1, 0].scatter(x, y1, color="red")
19 axs[1, 0].set_title("Scatter Plot")
20
21 # Quarto grafico
22 axs[1, 1].hist(y1, bins=5, color="purple")
23 axs[1, 1].set_title("Istogramma")
24
25 plt.tight_layout() # Migliora la disposizione
26 plt.show()
27
```


Matplotlib – Subplots

Grafico Lineare

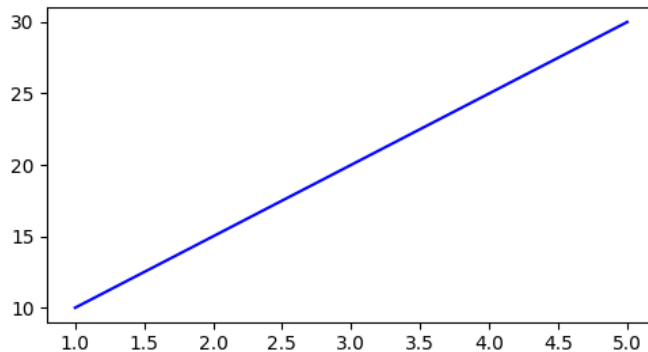
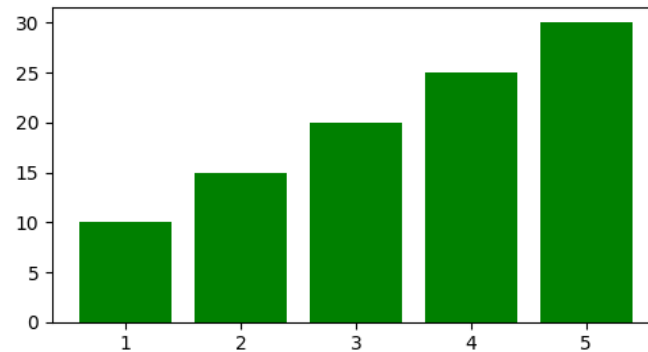
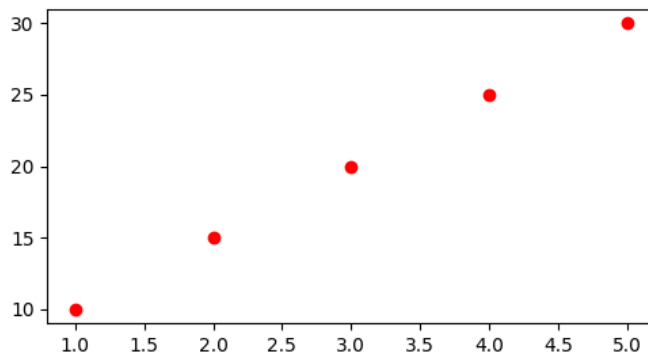


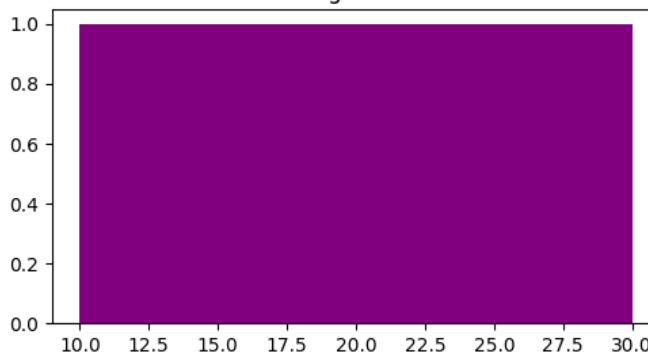
Grafico a Barre



Scatter Plot



Istogramma



Matplotlib – Save chart as image

main.py X

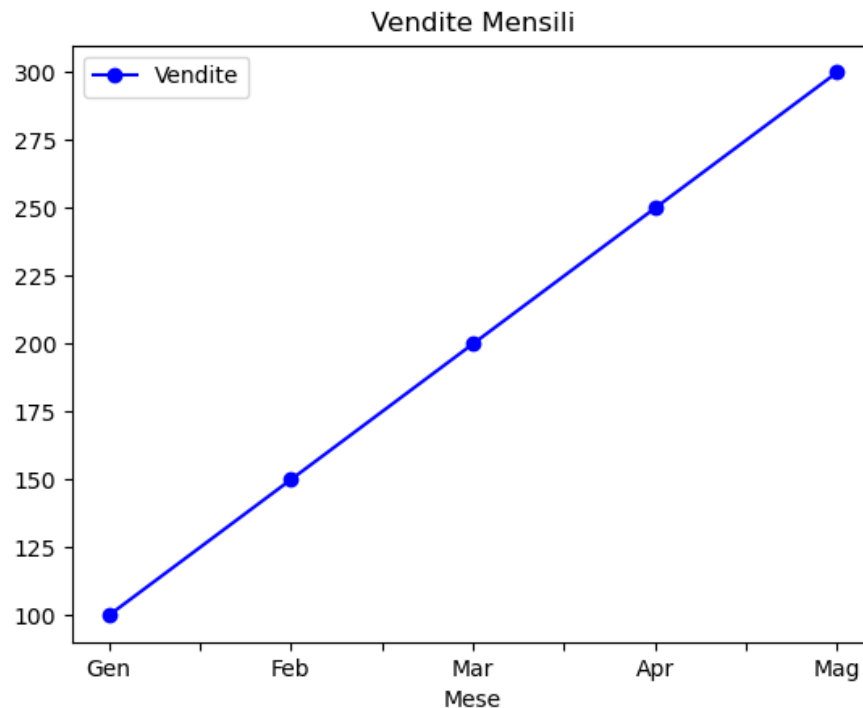
main.py > ...

```
1  import matplotlib.pyplot as plt
2
3  x = [1, 2, 3, 4, 5]
4  y1 = [10, 15, 20, 25, 30]
5  y2 = [5, 10, 15, 20, 25]
6
7  plt.plot(x, y1, label="Linea 1")
8  plt.title("Salvataggio del Grafico")
9  plt.savefig("grafico.png") # Salva l'immagine
10 plt.show()
11
```

Matplotlib – Matplotlib and pandas

```
main.py ×  
main.py > ...  
1  import matplotlib.pyplot as plt  
2  import pandas as pd  
3  
4  
5  # Creazione di un DataFrame  
6  df = pd.DataFrame({  
7      "Mese": ["Gen", "Feb", "Mar", "Apr", "Mag"],  
8      "Vendite": [100, 150, 200, 250, 300]  
9  })  
10  
11 # Creazione del grafico con Pandas  
12 df.plot(x="Mese", y="Vendite", kind="line", marker="o", color="blue", title="Vendite Mensili")  
13  
14 plt.show()  
15
```

Matplotlib – Matplotlib and pandas



Exercises

1. **Checking whether a Number is Even or Odd**

Write a program that asks the user to enter a number and then prints whether it is even or odd.

2. **Sum of Numbers from 1 to N**

Write a program that asks the user for a number N and calculates the sum of all numbers from 1 to N.

3. **Find the Largest Number in a List**

Write a program that takes a list of numbers and finds the maximum value without using `max()`.

Exercises

4. Create a dataframe and save it as csv

Mese	Prodotto	Quantità	Vendite
Gennaio	Telefono	50	25000
Gennaio	Laptop	30	60000
Febbraio	Telefono	40	20000
Febbraio	Laptop	35	70000
Marzo	Telefono	60	30000
Marzo	Laptop	45	90000

Exercises

5. **Create a bar chart to compare sales for each month.**